MCIR: A Canonical Intermediate Representation for Governed Transformation

Hash-Stable Structure for Semantics, Authority, and Admissible Change

A change to admissibility, authority scope, transformation semantics, or capability exposure must therefore alter the canonical structure.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Modern computation increasingly depends on transformations whose authority and consequences are not fully captured by source code, binaries, policies, or configuration files alone. Textual code is a flattened projection of abstract structure, while binary code is a still more opaque target realization. Neither representation natively records which changes alter admissibility, authority scope, transformation semantics, or permitted effects.

This paper introduces MCIR, Meta-Compilation Intermediate Representation, as a canonical, hash-stable representation for governed transformation. MCIR is not merely an intermediate representation for programs; it is an intermediate representation for systems of structured change, including compilers, policies, protocols, capability regimes, semantic governance layers, and other transformation-bearing systems. Its core property is canonicalization: any admitted definition has exactly one valid structural normal form under a closed, versioned grammar and deterministic normalization rules.

MCIR requires governance-relevant parameters to be represented as first-class structural nodes. A change to admissibility, authority scope, transformation semantics, or capability exposure must therefore alter the canonical structure. This makes governance-relevant divergence mechanically visible, diffable, and hash-identifiable.

We formalize governance relevance as a predicate over parameters that can alter admissibility, authority relations, transformation semantics, or capability exposure. We define admissibility as a first-class function, distinguish grammar-scoped structural equivalence from canonical form, state the key canonicalization invariants, and provide worked examples involving threshold changes, authority expansion, unordered capability sets, and a fully formal minimal grammar

fragment. We further clarify that hashing is compressed identity over canonical bytes, not the source of canonical uniqueness, and propose deterministic collision containment through cascading domain-separated identifiers.

MCIR provides a representation discipline for AI-native and governance-aware computation: before a system is lowered into code, binary, policy execution, protocol behavior, or runtime effect, its admissible structure can be made explicit, canonical, and mechanically comparable.

---

1. Introduction

Computing systems evolve through transformation.

Source lowers to binary. Policies constrain execution. Protocols define message exchange. Optimizers alter structure while attempting to preserve behavior. Configuration changes expose or restrict capability. Governance arrangements shift who may mutate, override, admit, refuse, or execute.

These transformations are consequential, yet their definitions are typically fragmented across source code, documentation, configuration files, build scripts, deployment pipelines, policy engines, runtime flags, and convention. A governance-relevant change may occur without a single canonical structural locus where that change must appear. Thresholds can shift. Authority scope can expand. Capability exposure can widen. A transformation may produce different outputs for identical inputs. The system changes, but the evidence of that change remains distributed.

MCIR adopts a single architectural principle:

If a system changes in governance-relevant ways, that change must alter its canonical structural representation.

Formally, for MCIR artifacts X and Y under grammar version G_v:

$$\begin{aligned} &\text{GovRelDiff}_{G_v}(X, Y) \\ &\Rightarrow \text{canonicalize}_{G_v}(X) \\ &\neq \text{canonicalize}_{G_v}(Y) \end{aligned}$$

MCIR does not define correct governance. It defines whether governance has changed.

This distinction matters. In ordinary software systems, one can often change the behavior of a system without changing the visible form that humans treat as authoritative. A configuration file may alter a policy. A build flag may change an optimizer. A runtime capability may be exposed through a deployment setting. A special authority may be introduced through an exception path. These changes may be operationally decisive while remaining structurally underrepresented.

MCIR is designed to prevent such invisibility.

It does so by representing governance-bearing transformation as a closed, versioned, canonical abstract syntax tree. Admissibility rules, authority bindings, capability exposure, transformation semantics, and other governance-relevant parameters must be explicit nodes or fields within the representation. The canonical representation can then be normalized, diffed, hashed, and compared. Formatting, textual order, narrative explanation, and incidental serialization differences cannot mask governance-relevant divergence.

This paper presents MCIR as a canonical representation for governed transformation.

---

2. AI-Generated Code and the Limits of Textual Representation

Recent discussion around AI-generated software often frames the future as a choice between textual code and direct binary generation. This framing misses the more important representation problem.

Textual code is already a flattened projection of deeper structure. It serializes abstract syntax, binding relationships, control flow, data flow, authority assumptions, and effect boundaries into a linear token stream. Humans have learned to read and write that stream. Compilers have learned to parse it back into structure. Large language models can generate it token by token. But textual code is not the structure itself.

Binary code is a still more compressed and target-specific projection. It may be executable, but it is poorly suited for expressing why a transformation is admissible, what authority it requires, what capabilities it exposes, which claims it supports, or what semantic continuity it preserves.

Direct binary generation therefore moves in the wrong direction for governance-aware systems. It may produce executable artifacts, but it maximizes opacity at precisely the layer where authority, semantics, and admissibility must become explicit.

MCIR proposes a different path. AI systems may propose transformations, but those transformations should be represented in a canonical governed structure before they are lowered into textual code, backend IR, binary output, policy execution, or runtime effects. In this architecture, the relevant question is not merely whether an artifact runs. The question is whether the artifact's admissible structure is explicit, comparable, and bound to the authority and evidence that permit it to move.

MCIR is therefore not a replacement for all programming languages or compiler IRs. It is a canonical representation layer for governed transformation. Textual languages, domain-specific human-legible IRs, compiler IRs, and binaries may

all serve as projections or lowerings. MCIR is the structure that prevents governance-relevant meaning from being lost in those projections.

---

3. Formal Preliminaries

Let S be a state space.

Let I be an input space.

Let O_A be an admissibility outcome space.

Let E be a set of entities.

Let R be a set of resources.

Let P be a set of powers or permissions.

Let C_all be a universe of capabilities.

MCIR uses four semantic surfaces to define governance relevance:

1. Admissibility.
2. Authority.
3. Transformation.
4. Capability.

Admissibility is modeled as a function:

$$A : S \times I \to O_A$$

For example, a domain may define:

$$O_A = \{\text{admit}, \text{reject}, \text{refuse}, \\ \text{defer}, \text{unsupported}\}$$

Authority is modeled as a relation:

$$\Sigma \subseteq E \times R \times P$$

Transformation semantics are modeled as a state-and-output transition:

$$T : S \times I \to S \times O_T$$

Capability exposure is modeled as a subset of available capabilities:

$$C \subseteq C_{\text{all}}$$

A parameter p is governance-relevant when changing p can change at least one of these four surfaces.

$$\mathrm{GovRel}(p) \iff \exists s, i : \Delta_p(A, \Sigma, T, C) \neq 0$$

Equivalently:

$$\begin{aligned}\mathrm{GovRel}(p) \iff & \Delta_p A \\ & \neq 0 \\ & \vee \Delta_p \Sigma \\ & \neq 0 \\ & \vee \Delta_p T \\ & \neq 0 \\ & \vee \Delta_p C \\ & \neq 0\end{aligned}$$

MCIR imposes the structural representation obligation:

$$\mathrm{GovRel}(p) \Rightarrow \mathrm{StructRep}(p)$$

Every governance-relevant parameter must be structurally represented within the MCIR artifact.

Failure to represent such a parameter is representation unsoundness. It is not a canonicalization failure. Canonicalization can only normalize the structure it is given.

This distinction is central. MCIR cannot repair a model that hides governance outside the representation. MCIR can make such hiding a violation of the representation contract.

---

4. Governance-Relevant Parameters

Operationally, a parameter is governance-relevant if it influences any of the following:

1. Admissibility: accept, reject, admit, refuse, validate, invalidate, or classify decisions.
2. Authority scope: who or what may mutate, override, admit, execute, certify, or bind rules.
3. Transformation semantics: outputs, state transitions, or behavior for identical inputs.

4. Capability exposure: permitted effects, accessible resources, execution powers, or environmental reach.

The operational boundary rule is:

Any behavior that can change acceptance or rejection, authority to mutate or override, transformation outputs for identical inputs, or permitted effects must be represented as nodes within Definitions, Constraints, Transformations, or Bindings.

External configuration is permitted only for governance-irrelevant parameters, such as display labels or non-semantic formatting preferences, and those parameters must be confined to Metadata.

MCIR does not automatically infer governance relevance. Designers must declare it explicitly and must satisfy the structural representation obligation. Undeclared governance relevance is representation unsoundness.

Examples of governance-relevant parameters include:

```
admission thresholds
refusal predicates
authority keys
override rules
capability lists
effect permissions
transformation functions
precision rules
runtime target constraints
binding scopes
policy version
grammar version
```

Examples of governance-irrelevant metadata may include:

```
display title
human-facing description
editor fold preference
UI color label
documentation note
```

However, a field is not governance-irrelevant merely because it is stored in a configuration file or described as metadata. If it changes admissibility, authority, transformation outputs, or permitted effects, it must be represented within the governance-bearing structure.

---

5. Admissibility as a First-Class Function

Admissibility is not merely an informal label in MCIR. It is a first-class semantic function.

Let:

$$O_A = \{\text{admit}, \text{reject}, \text{refuse}, \\ \text{defer}, \text{unsupported}\}$$

Then:

$$A : S \times I \to O_A$$

maps a state and input to an admissibility outcome.

Constraints are partial structural definitions of A. Thresholds, refusal predicates, authority requirements, and capability gates all contribute to determining A.

For example, a threshold rule:

$$\text{score}(i) \geq \text{minimum_score}$$

may define part of A as follows:

$$\text{score}(i) \geq \text{minimum_score} \Rightarrow A(s, i) = \text{admit}$$

$$\text{score}(i) < \text{minimum_score} \Rightarrow A(s, i) = \text{refuse}$$

This avoids Pandoc alignment environments while preserving the intended formal statement.

Authority bindings may also define A. If an input proposes a transformation outside the authority scope of the signer or actor, A may return refuse regardless of other constraints.

Capability exposure may also define A. If an input requires a capability absent from C, the input may be refused or classified unsupported.

Thus the main MCIR regions relate to admissibility as follows:

```
Definitions define objects, thresholds, authorities, and capabilities.
Constraints define predicates over state, input, and definitions.
Transformations define state and output change.
Bindings define who or what may apply or mutate definitions,
constraints, and transformations.
Metadata may not alter A.
```

A valid MCIR artifact need not contain a fully executable decision procedure for A in every domain. However, any structural component that can affect A must be represented.

This unifies thresholds, constraints, refusal predicates, and authority gates as structural contributors to admissibility.

---

6. Formal Structure of MCIR

MCIR is defined over a finite, acyclic, versioned grammar.

A schematic grammar is:

```
MCIR ::= Header Definitions Constraints Transformations Bindings
Metadata

Header ::= version identifier domain

Definitions ::= Definition*
Definition ::= ObjectDef
             | InvariantDef
             | ThresholdDef
             | AuthorityDef
             | CapabilityDef

Constraints ::= Constraint*
Constraint ::= Expr

Transformations ::= Transformation*
Transformation ::= name inputs outputs semantics

Bindings ::= Binding*
Binding ::= AuthorityBinding
          | ContextBinding

Metadata ::= key value*
```

The top-level structure has six major regions.

6.1 Header

The Header defines the grammar version, identifier, and domain. Versioning is governance-relevant. A change in grammar version changes the rules of parsing, normalization, and interpretation. It must therefore be explicit.

6.2 Definitions

Definitions introduce named governance-bearing objects. These include objects, invariants, thresholds, authorities, and capabilities. The exact node set is closed per grammar version.

6.3 Constraints

Constraints define admissibility predicates, refusal conditions, structural restrictions, or other governing expressions. These expressions are represented as expression trees within the same grammar version.

6.4 Transformations

Transformations define structured change. A transformation includes inputs, outputs, and semantics. MCIR does not require all transformations to be executable in the same sense as a programming language function. It requires their governance-relevant structure to be explicit.

6.5 Bindings

Bindings connect authority, context, scope, and transformations. For example, an AuthorityBinding may state that a particular authority may mutate Constraints but not Transformations, or may admit a certain class of object but not override capability exposure.

6.6 Metadata

Metadata contains governance-irrelevant annotations. Metadata may support human use, documentation, display, or indexing, but it must not influence admissibility, authority, transformation semantics, or capability exposure.

The grammar obeys the following properties:

```
closed per version
finite
acyclic
no implicit defaults
explicit ordered/unordered child declarations
deterministic normalization rules
```

No implicit defaults are permitted. If a field is required, it must be present. If a field is optional, the absence itself must have explicit grammar-defined meaning. Hidden defaults are a common source of governance drift and are therefore disallowed.

Each node declares whether its children are ordered, unordered, or multiset-like. This prevents textual order from becoming accidentally governance-relevant or accidentally governance-irrelevant.

7. Grammar-Scoped Structural Equivalence

Structural equivalence is always scoped to a grammar version.

Two MCIR ASTs X and Y are structurally equivalent under grammar version G_v, written:

$$X \equiv_{G_v} Y$$

if the following conditions hold:

```
node types match under G_v
all governance-relevant fields match after normalization under G_v
ordered children are equal as sequences
unordered children are equal as multisets under structural equivalence
identifiers are alpha-normalized according to G_v
numeric literals are normalized under declared precision in G_v
expression trees match under declared structural rules in G_v
metadata differences are ignored only where G_v declares them
governance-irrelevant
```

Structural equivalence is not broad semantic equivalence.

MCIR does not assume that two different expressions are equivalent merely because they may be algebraically reducible to the same value. For example:

```
x + y
```

and:

```
y + x
```

are structurally equivalent only if G_v explicitly declares the operator commutative and canonicalizes the operands accordingly.

Similarly:

```
(x + y) + z
```

and:

```
x + (y + z)
```

are structurally equivalent only if G_v explicitly declares associativity for the relevant expression form.

This restriction is intentional. Broad semantic equivalence is difficult, domain-dependent, and often undecidable. MCIR's goal is not to solve all semantic equivalence. Its goal is to define a canonical structural representation for declared governance semantics.

The equivalence relation is therefore grammar-scoped:

$$\equiv_{G_v}$$

It is not a universal relation independent of grammar version.

Structural equivalence is defined independently of canonicalization. Canonicalization computes a representative element of each grammar-scoped structural equivalence class.

---

8. Canonical Form

A canonical form is a representative element of each structural equivalence class under G_v.

Canonicalization is a function:

$$\text{canonicalize}_{G_v} : AST_{G_v} \to AST_{G_v}$$

such that the following hold.

Soundness:

$$X \equiv_{G_v} Y \Rightarrow \text{canonicalize}_{G_v}(X) = \text{canonicalize}_{G_v}(Y)$$

Completeness:

$$\text{canonicalize}_{G_v}(X) = \text{canonicalize}_{G_v}(Y) \Rightarrow X \equiv_{G_v} Y$$

Idempotence:

$$\text{canonicalize}_{G_v}(\text{canonicalize}_{G_v}(X)) = \text{canonicalize}_{G_v}(X)$$

Thus canonicalization computes a normal form for grammar-scoped structural equivalence.

The canonical form is not a claim of total semantic truth. It is not an assertion that two systems behave identically under every possible runtime condition. It is not a proof that an implementation matches an intention. It is a structural normal form for declared governance semantics under a specified grammar.

The phrase “canonical identity” in MCIR therefore refers to the canonical bytes of the normalized structure, not to metaphysical uniqueness or complete semantic equivalence.

---

9. Core Invariants

This section states the core invariants of MCIR in theorem-style form.

9.1 Governance Visibility Invariant

For any two valid MCIR artifacts X and Y under grammar version G_v:

$$\begin{aligned}&\text{GovRelDiff}_{G_v}(X, Y)\\ &\Rightarrow \text{canonicalize}_{G_v}(X)\\ &\neq \text{canonicalize}_{G_v}(Y)\end{aligned}$$

Meaning: governance-relevant divergence must alter canonical structure.

9.2 Termination Invariant

For every valid finite acyclic AST under G_v:

$$\text{canonicalize}_{G_v}(X)$$

terminates.

This follows from finite recursion over an acyclic tree, deterministic normalization at each node, and finite sorting over finite child sets.

9.3 Idempotence Invariant

For every valid AST X under G_v:

$$\text{canonicalize}_{G_v}(\text{canonicalize}_{G_v}(X)) = \text{canonicalize}_{G_v}(X)$$

Meaning: once a structure is canonical, repeated canonicalization does not change it.

9.4 Soundness Invariant

For valid ASTs X and Y under G_v:

$$X \equiv_{G_v} Y \Rightarrow \text{canonicalize}_{G_v}(X) = \text{canonicalize}_{G_v}(Y)$$

Meaning: structurally equivalent artifacts normalize to the same canonical form.

9.5 Completeness Invariant

For valid ASTs X and Y under G_v:

$$\text{canonicalize}_{G_v}(X) = \text{canonicalize}_{G_v}(Y) \Rightarrow X \equiv_{G_v} Y$$

Meaning: identical canonical forms imply structural equivalence under the grammar version.

9.6 Version-Scoped Uniqueness Invariant

Canonical uniqueness is scoped to G_v. If the grammar version changes, the equivalence relation and canonicalization function may also change.

$$\begin{aligned} & G_v \\ & \neq G_w \\ & \Rightarrow \equiv_{G_v} \text{ need not equal } \equiv_{G_w} \end{aligned}$$

Thus canonical identity is always interpreted relative to the grammar version that produced it.

---

10. Canonicalization Algorithm

A schematic canonicalization algorithm is:

```
function canonicalize_Gv(node):
    assert node conforms to G_v
    normalize_numeric_literals_Gv(node)
    normalize_identifiers_Gv(node)
    expand_all_fields_explicit_Gv(node)
    for child in node.children:
        canonicalize_Gv(child)
    if node.has_unordered_children:
        sort(node.children, key=structural_hash_Gv)
    sort_fields_Gv(node)
    return node
```

Where:

$$\text{structural_hash}_{G_v}(\textit{child}) = H(\text{emit}_{G_v}(\text{canonicalize}_{G_v}(\textit{child})))$$

and emit is the canonical serialization function for grammar version G_v.

The algorithm proceeds bottom-up. Children are canonicalized before unordered child sets are sorted. Sorting is deterministic and based on canonical subtrees. Field ordering is deterministic and grammar-defined.

10.1 Numeric Normalization

Numeric literals are normalized according to declared precision rules. If a domain requires integer precision, floating representations cannot be silently substituted. If a domain permits decimal precision, rounding and formatting rules must be explicit.

10.2 Identifier Normalization

Identifiers are alpha-normalized where permitted. This means that purely local names may be normalized without changing structure. However, authority identifiers, external references, cryptographic keys, and domain-significant identifiers are not alpha-normalized unless the grammar explicitly declares them local and non-governance-relevant.

10.3 Explicit Field Expansion

All fields are expanded explicitly. No hidden defaults remain. If an empty list is the default, the canonical form contains the empty list. If absence is meaningful, absence is represented according to grammar rules.

10.4 Ordered and Unordered Children

Ordered children remain ordered. Unordered children are sorted by canonical structural hash or by a grammar-defined total ordering. This ensures that textual or serialization order cannot alter identity where the grammar declares order irrelevant.

10.5 Well-Founded Structural Hashing

The use of structural hash for sorting unordered children creates an apparent dependency:

```
canonicalization uses hashing for sorting
hashing uses canonicalization for canonical bytes
```

This dependency is well-founded because the AST is finite and acyclic and because canonicalization is bottom-up.

For any node n, all children of n are canonicalized before n sorts its unordered children. Therefore the structural hashes used for sorting children of n are computed over already canonicalized child subtrees. No parent hash is required to canonicalize a child. No child depends on the canonical form of its parent. Thus no cyclic dependency exists.

---

11. Termination and Uniqueness Sketch

Given:

```
finite acyclic AST
closed grammar per version
deterministic normalization
explicit ordered/unordered child declaration
bottom-up canonicalization
deterministic structural hashing
canonical serialization
```

canonicalization terminates.

The proof sketch is straightforward. Because the AST is finite and acyclic, recursive traversal terminates. Each node is processed a finite number of times. Normalization functions are deterministic and total over valid nodes. Sorting unordered children over a finite set terminates. Therefore canonicalization terminates for all valid ASTs.

Canonical form is unique per structural equivalence class because all sources of representational variation included in structural equivalence are normalized deterministically:

```
identifier variation
numeric literal variation
field order variation
unordered child order variation
explicit default variation
declared expression normalization
```

Soundness follows because structurally equivalent ASTs normalize every equivalence-preserving variation to the same representation.

Completeness follows because equal canonical forms must share the same normalized node types, fields, child structures, identifiers, numeric values, and declared expression structures; therefore the original ASTs are structurally equivalent under the definition of equivalence scoped to G_v.

Idempotence follows because once an AST is in canonical form, applying the same deterministic normalization rules again produces the same AST.

---

12. Canonical Serialization and Hash Identity

MCIR uses hashing as compressed identity, not as the source of canonical uniqueness.

Let:

$$S = \text{emit}_{G_v}(\text{canonicalize}_{G_v}(AST))$$

where S is the canonical serialized byte sequence.

Then:

$$\text{MCIR_ID}_1 = H_1(S)$$

The canonical identity is the canonical form and canonical bytes. The hash is a compact identifier over those bytes.

This distinction matters because cryptographic hashes are collision-resistant, not mathematically injective. MCIR does not require hash injectivity for canonicalization. Canonicalization is provided by grammar and normalization rules. Hashing provides compact reference and practical collision resistance.

12.1 Collision Containment

If a collision is observed for MCIR_ID_1, meaning two distinct canonical serializations share the same digest, MCIR permits deterministic refinement:

$$\text{MCIR_ID} = (H_1(S), H_2(S), H_3(S), ...)$$

Where each H_i is a domain-separated hash function over the same canonical bytes:

$$H_i(S) = H(\text{domain_separator}_i \| S)$$

This refinement is:

```
deterministic in S
not user-generated
not context-dependent
extended only upon observed ambiguity
compatible with compact default identifiers
```

Collision detection requires access to canonical bytes S or a trusted retrieval path for comparing canonical bytes when an identifier conflict appears.

MCIR therefore distinguishes:

```
canonical identity: canonical form and canonical bytes
compressed identifier: hash tuple used for compact reference
```

This avoids the common error of treating hashes as ontological identity.

---

13. Diff Semantics

Structural diff compares canonical ASTs.

A formal sketch:

$$\Delta(X, Y) = \text{TreeEdit}(\text{canonicalize}_{G_v}(X), \text{canonicalize}_{G_v}(Y))$$

The raw delta is a tree-edit structure over canonical forms.

MCIR then classifies structural deltas through a mapping:

$$\phi : \Delta \to \text{GovernanceChangeClass}$$

Examples of GovernanceChangeClass include:

```
threshold change
authority expansion
authority contraction
capability enablement
capability removal
transformation semantic change
constraint addition
constraint removal
binding addition
binding removal
grammar version change
metadata-only change
```

A schematic diff procedure is:

```
diff_Gv(X, Y):
    X_c = canonicalize_Gv(X)
    Y_c = canonicalize_Gv(Y)
    delta = TreeEdit(X_c, Y_c)
    classes = phi(delta)
    return delta, classes
```

Formatting and textual order cannot mask divergence. If a governance-relevant parameter changes, the canonical AST changes. If the canonical AST changes, structural diff can classify the change.

This enables a governance-aware review process. Instead of asking only "what changed in the text?", reviewers can ask:

```
what changed in admissibility?
what changed in authority?
what changed in capability exposure?
what changed in transformation semantics?
```

```
what changed only in metadata?
```

This is the difference between syntactic review and governed structural review.

---

14. MCIR as a Semantic Commit Object

MCIR may be understood as a semantic commit object.

In ordinary version control, a commit records a content state, parent relation, author metadata, and a hash identifier. It helps make textual project history visible and referable.

MCIR generalizes this logic to governed transformation.

In this framing:

```
canonical form = commit content
canonical bytes = committed structural object
hash tuple = compact commit identifier
grammar version = commit schema
structural diff = semantic delta
authority bindings = admissibility context
capability definitions = effect scope
constraints = admissibility surface
transformations = governed change semantics
```

Unlike an ordinary code commit, an MCIR artifact is not merely a snapshot of files. It is a canonical commitment to a governance-bearing structure.

This framing clarifies why canonicalization matters. Without canonical form, superficial representation differences can masquerade as structural change, or structural change can hide behind incidental formatting. With canonical form, the semantic commit object becomes mechanically comparable.

MCIR does not replace version control. It supplies a canonical structure that version control can store, reference, review, and distribute.

---

15. Worked Example: Threshold-Based Admission

Consider an admission policy.

15.1 Variant A

Admission rule:

```
Accept if score >= 75.
Authority: OversightBoard.
```

Schematic MCIR:

```
Header:
  version: 1
  identifier: AdmissionPolicy
  domain: access_control

Definitions:
  ThresholdDef:
    name: minimum_score
    value: 75

  AuthorityDef:
    name: OversightBoard
    key: PUBKEY_ABC

Constraints:
  score >= minimum_score

Bindings:
  AuthorityBinding:
    authority: OversightBoard
    scope: Constraints
```

Canonical serialization, schematic:

```
{
  domain: access_control,
  identifier: AdmissionPolicy,
  version: 1,
  thresholds: [
    { name: minimum_score, value: 75 }
  ],
  authorities: [
    { name: OversightBoard, key: PUBKEY_ABC }
  ],
  constraints: [
    score >= minimum_score
  ],
  bindings: [
    { authority: OversightBoard, scope: Constraints }
  ]
}
```

Hash:

$$H_A = H(serialization_A)$$

15.2 Variant B: Threshold Raised

Variant B changes:

```
ThresholdDef.minimum_score.value: 75 -> 80
```

This changes admissibility. Therefore it is governance-relevant.

The canonical structure changes. The diff reports:

```
ThresholdDef.minimum_score.value: 75 -> 80
```

And:

$$H_B \neq H_A$$

15.3 Variant C: Authority Expansion

Variant C adds:

```
AuthorityDef:
  name: EmergencyOverrideBoard
  key: PUBKEY_DEF

AuthorityBinding:
  authority: EmergencyOverrideBoard
  scope: Transformations
```

This expands authority scope. Therefore it is governance-relevant.

The canonical structure changes. The diff reports:

```
AuthorityDef added: EmergencyOverrideBoard
AuthorityBinding added: EmergencyOverrideBoard -> Transformations
```

And:

$$H_C \neq H_A$$

15.4 Variant D: Unordered Capability Set

Suppose capabilities are declared unordered.

Variant D1:

```
CapabilityDef:
  name: allowed_effects
  values: {net, fs}
```

Variant D2:

```
CapabilityDef:
  name: allowed_effects
  values: {fs, net}
```

Because values is declared unordered, canonicalization sorts the set under the grammar-defined order:

`values: {fs, net}`

Therefore:

$$\text{canonicalize}_{G_v}(D1) = \text{canonicalize}_{G_v}(D2)$$

and:

$$H_{D1} = H_{D2}$$

Textual order does not affect canonical representation when order is declared governance-irrelevant.

15.5 Variant E: Capability Enablement

Now suppose Variant E changes:

`values: {fs}`

to:

`values: {fs, net}`

This enables network capability. Capability exposure is governance-relevant. Therefore the canonical structure changes.

Diff:

```
CapabilityDef.allowed_effects.values: +net
```

And:

$$H_E \neq H_D$$

---

16. Fully Formal Minimal Example

This section gives a compact formal example with a tiny grammar fragment.

Let G_1 define:

```
Policy ::= Header Thresholds Capabilities

Header ::= version identifier

Thresholds ::= Threshold*
Threshold ::= name value

Capabilities ::= CapabilitySet
CapabilitySet ::= { Capability* }

Capability ::= fs | net | exec
```

Assume:

```
Thresholds is an unordered set keyed by name.
Capabilities is an unordered set.
Header fields are ordered by grammar rule: version, identifier.
Numeric values are integers.
No implicit defaults are permitted.
```

Define AST X:

```
Policy(
  Header(version=1, identifier="P"),
  Thresholds[
    Threshold(name="minimum_score", value=75),
    Threshold(name="maximum_retries", value=3)
  ],
  Capabilities{net, fs}
)
```

Define AST Y:

```
Policy(
  Header(identifier="P", version=1),
  Thresholds[
    Threshold(name="maximum_retries", value=3),
    Threshold(name="minimum_score", value=75)
  ],
  Capabilities{fs, net}
)
```

Under G_1:

```
Header field order is normalized.
Thresholds is unordered.
Capabilities is unordered.
```

Therefore:

$$X \equiv_{G_1} Y$$

Canonicalization steps for X:

```
1. Confirm X conforms to G_1.
2. Expand all required fields explicitly.
3. Normalize Header field order to version, identifier.
4. Canonicalize Threshold children.
5. Sort Thresholds by canonical key or structural hash.
6. Sort Capabilities.
7. Emit canonical serialization.
```

Canonicalization steps for Y are identical after normalization.

Canonical serialization for both:

```
Policy{
  Header{
    version:1,
    identifier:"P"
  },
  Thresholds:[
    Threshold{name:"maximum_retries",value:3},
    Threshold{name:"minimum_score",value:75}
  ],
  Capabilities:{fs,net}
}
```

Thus:

$$\text{canonicalize}_{G_1}(X) = \text{canonicalize}_{G_1}(Y)$$

and:

$$H(\text{emit}_{G_1}(\text{canonicalize}_{G_1}(X))) = H(\text{emit}_{G_1}(\text{canonicalize}_{G_1}(Y)))$$

Now define AST Z:

```
Policy(
  Header(version=1, identifier="P"),
  Thresholds[
    Threshold(name="minimum_score", value=80),
    Threshold(name="maximum_retries", value=3)
  ],
  Capabilities{fs, net}
)
```

Z differs from X by minimum_score:

```
75 -> 80
```

Because minimum_score participates in admissibility, GovRel(minimum_score) holds.

Therefore:

$$X \not\equiv_{G_1} Z$$

and:

$$\text{canonicalize}_{G_1}(X) \neq \text{canonicalize}_{G_1}(Z)$$

Diff:

$$\Delta(X, Z) = \text{TreeEdit}(\text{canonicalize}_{G_1}(X), \text{canonicalize}_{G_1}(Z))$$

Classification:

$$\phi(\Delta(X, Z)) = \text{threshold_change}$$

This example illustrates the essential discipline: representational variation disappears, governance-relevant variation remains.

---

17. Worked Example: Transformation Semantics

Consider a transformation that maps input records into an output classification.

Variant A:

```
Transformation:
  name: classify
  inputs: [score]
  outputs: [status]
  semantics:
    if score >= minimum_score:
      status = accepted
    else:
      status = refused
```

Variant B changes the refusal branch:

```
else:
  status = manual_review
```

For identical inputs below threshold, the output changes. Therefore this is governance-relevant transformation divergence.

MCIR records the transformation semantics structurally. The diff reports:

```
Transformation.classify.semantics.false_branch:
  refused -> manual_review
```

This is not merely an implementation change. It changes the admissibility consequence of the policy.

Variant C changes only a display label:

```
Metadata:
  refused_label: "Denied"
```

to:

```
Metadata:
  refused_label: "Not admitted"
```

If the grammar declares this label governance-irrelevant, canonical governance identity does not change. Metadata identity may change, but structural governance identity remains the same.

This distinction allows MCIR to separate governance-bearing transformation from human-facing annotation.

---

18. Versioning and Extensibility

The grammar is closed per version.

New node types require a version increment. Older versions remain parseable and canonicalizable under their own rules. Canonicalization rules are bound to grammar version.

Versioning itself is governance-relevant. A change from one grammar version to another may change what can be represented, what must be explicit, or how equivalence is determined. Therefore grammar version appears in the Header and participates in canonical identity.

Versioning rules include:

```
grammar version must be explicit
canonicalization rules are version-bound
old versions remain canonicalizable
new node types require version increment
migration must be represented as a transformation
version change is diff-visible
```

MCIR does not allow silent grammar evolution. If representation rules change, that change must itself be visible.

---

19. Relationship to Existing Theory and Systems

MCIR draws from several established traditions, but combines them for a different purpose.

19.1 Term Rewriting and Normal Forms

MCIR uses the idea of a normal form: structurally equivalent artifacts reduce to a unique canonical representative. This resembles term rewriting systems where expressions are rewritten into normal forms under a rewrite relation.

MCIR differs in that its rewrite system is not designed primarily for computation reduction or algebraic simplification. It is designed for governance-bearing structural commitment. Rewrites are limited to grammar-declared normalization rules.

19.2 Alpha-Equivalence in Lambda Calculus

MCIR uses alpha-normalization where local names are not governance-relevant. This parallels alpha-equivalence in lambda calculus, where bound variable names may differ without changing structure.

MCIR restricts alpha-normalization to grammar-declared local identifiers. Authority names, cryptographic keys, external references, and domain-significant identifiers cannot be normalized away unless explicitly declared safe.

19.3 Abstract Interpretation

Abstract interpretation maps concrete program behavior into abstract domains for analysis. MCIR similarly represents governance-relevant aspects of a system in a structured abstraction.

The difference is that MCIR is not merely an analysis result. It is a canonical commitment object. It does not summarize behavior alone; it records the declared admissibility, authority, capability, and transformation structure to which other evidence may refer.

19.4 Proof-Carrying Code and Typed IRs

Proof-carrying code and typed IRs attach machine-checkable guarantees or type structure to executable artifacts. MCIR shares the goal of making properties explicit before execution.

MCIR differs by focusing on governed transformation. It represents authority scope, capability exposure, admissibility, refusal, and semantic change structure.

Proofs may be attached to or referenced by MCIR artifacts, but MCIR is not itself a proof system.

19.5 Content-Addressable Structures and Merkle DAGs

Content-addressable systems identify artifacts by hashes of their content. Merkle DAGs allow efficient integrity checking over structured graphs.

MCIR uses content addressing, but treats hashing as compressed identity over canonical bytes. It adds governance relevance, equivalence rules, canonicalization obligations, and diff classifications that ordinary content-addressed storage does not define.

19.6 Canonical JSON or CBOR

Canonical JSON and CBOR canonicalize encoding. They can ensure that the same data structure serializes the same way. They do not define governance relevance, structural equivalence classes, authority bindings, capability exposure, or diff semantics for admissibility-bearing change.

MCIR may use canonical serialization techniques, but its main contribution is not serialization. Its contribution is governance-aware structural representation.

19.7 Compiler IRs

Compiler IRs represent programs for analysis and lowering. They are usually optimized for transformation, analysis, or machine code generation. They do not usually represent authority scope, admissibility rules, governance bindings, or capability exposure as first-class constitutional commitments.

MCIR can relate to compiler IRs as a higher governance-bearing layer. Compiler IRs may be lowerings from MCIR-governed structures, or artifacts compared against MCIR definitions.

19.8 Policy-as-Code

Policy-as-code frameworks evaluate policies. They may define rules, constraints, and decisions. They do not necessarily require canonical structural equivalence, hash-stable normal forms, deterministic diff semantics, or explicit representation of all governance-relevant parameters as a closed grammar.

MCIR can represent policy systems, but it is not merely a policy evaluator.

19.9 Reproducible Builds

Reproducible build systems aim to ensure that the same inputs produce the same build outputs. They do not by themselves canonicalize authority bindings, admissibility semantics, or capability exposure as first-class nodes.

MCIR complements reproducible builds by representing governed transformation commitments before, during, and after lowering.

---

20. MCIR in a Governance-Aware Software Stack

In a broader constitutional computing stack, MCIR can serve as the canonical hub between human-legible specification, semantic decomposition, backend realization, and runtime custody.

A schematic pipeline:

```
HLIR / source / policy / protocol
-> GSIR or other structured semantic candidate representation
-> GMB staging
-> MCIR candidate preflight
-> MCIR admission where warranted
-> PEIR / CEIR / backend IR
-> package or binary realization
-> hosted runner or sandbox execution
-> custody receipt
-> MCRM comparison
-> standing or refusal report
```

In this stack:

```
HLIR provides human-legible governed proposal.
GSIR structures decomposed semantic candidates.
GMB stages candidate meaning without authority.
MCIR canonically represents admitted governed change.
PEIR / CEIR / backend IRs prepare execution-facing realization.
C-D.MS records custody and effects.
MCRM compares source, binary, runtime, and semantic surfaces.
```

MCIR does not need to contain all runtime evidence. It must, however, represent the canonical governed structure to which runtime evidence, custody receipts, and comparison records can refer.

This is how a system can distinguish:

```
source meaning
canonical governed meaning
backend realization
runtime behavior
custody evidence
semantic standing
```

without collapsing them into a single ambiguous artifact.

---

21. Limitations and Non-Claims

MCIR is intentionally limited.

It does not decide correct governance.

It does not infer undeclared governance relevance.

It does not prove broad semantic equivalence.

It does not guarantee that an implementation conforms to an MCIR definition.

It does not make a hash ontologically unique.

It does not replace compiler IRs, policy engines, proof systems, sandboxing systems, or runtime custody layers.

It does not eliminate the need for human judgment.

Its role is narrower:

```
make governed transformation explicit
canonicalize declared governance structure
make governance-relevant divergence mechanically visible
support structural diff
support hash-stable reference
separate canonical bytes from compressed identifiers
```

The most important non-claim is that canonical structural equivalence is not total semantic equivalence.

Two MCIR structures with the same canonical form are structurally equivalent under the declared grammar. Whether a particular implementation, binary, execution, or runtime behavior conforms to that structure is a separate evidentiary question.

Similarly, two systems may behave the same over a finite fixture set while differing structurally. MCIR does not erase such differences. It records structure. Other layers, such as runtime witnesses or comparison disciplines, may record bounded behavior.

---

22. Security and Failure Modes

MCIR is vulnerable to modeling errors. If a designer fails to represent a governance-relevant parameter, canonicalization cannot detect changes to that parameter. This is not a canonicalization failure. It is representation unsoundness.

Important failure modes include:

```
undeclared governance relevance
governance-bearing behavior hidden in metadata
```

```
implicit defaults
version drift
external configuration changing authority or effects
overbroad equivalence declarations
hash-only identity assumptions
non-deterministic canonicalization
ambiguous ordered/unordered child declarations
```

Mitigations include:

```
closed grammar
explicit no-implicit-default rule
governance relevance audit
metadata confinement
version-bound canonicalization
structural diff review
collision containment
retrievable canonical bytes
non-claims discipline
```

MCIR should be used with a validation surface that checks representation completeness relative to a domain's governance model. A valid canonical form is only as honest as the model boundary that produced it.

---

23. Future Work

Several extensions are natural.

23.1 Domain-Specific MCIR Profiles

Different domains may define profiles over the MCIR grammar:

```
software package governance
legal process governance
financial settlement governance
scientific workflow governance
AI-agent operation governance
protocol governance
compiler governance
```

Each profile can specify required node types, allowed transformations, authority models, precision rules, and admissibility structures.

23.2 MCIR-to-Backend Lowering

MCIR can be lowered into execution-facing forms, such as backend IRs, policy engines, sandbox profiles, or runtime packages. Such lowering must itself be

represented as a governed transformation.

23.3 Runtime Witness Binding

Runtime evidence can be bound back to MCIR definitions. A runtime witness does not prove total semantic equivalence, but it can support bounded claims about observed behavior under specified conditions.

23.4 Source-Binary-Runtime Correspondence

MCIR can serve as the canonical semantic hub for comparing source, binary, and runtime artifacts. Such correspondence must preserve distinctions among declared intent, compiled realization, observed behavior, and admitted standing.

23.5 AI-Native Transformation Review

AI systems can propose MCIR transformations directly. Review then occurs over canonical structural deltas rather than over free-form natural language summaries or flattened textual code alone.

---

24. Conclusion

MCIR extends intermediate representation from execution mechanics to governed transformation. By requiring governance-relevant parameters to appear as first-class structural nodes, defining grammar-scoped structural equivalence over those nodes, and computing canonical normal forms under a closed versioned grammar, MCIR ensures that admissibility-relevant change cannot hide behind formatting, ordering, configuration convention, or narrative explanation.

Canonical form provides structural uniqueness for declared governance semantics under a specified grammar. Cryptographic hashing provides compact collision-resistant identifiers over canonical bytes, while deterministic cascading identifiers contain observed hash ambiguity without making identity user-dependent.

MCIR does not decide what governance should be. It does not prove broad semantic equivalence, infer undeclared governance relevance, or replace backend execution systems. Its role is narrower and more fundamental: to make governed change explicit, canonical, diffable, and mechanically identifiable before that change is lowered into code, binary, protocol behavior, policy execution, or runtime effect.

In an AI-native software environment, this representation discipline becomes essential. Systems that can generate code or binaries faster than humans can review them require a substrate where authority, admissibility, capabilities, transformations, and claim boundaries are represented structurally.

Textual code is a flattened projection. Binary is target realization. MCIR is canonical governed structure.

That is the representation layer required for software whose changes cannot merely execute, but must be admissible.

Corpus-Wide Reference Positioning

This paper argues that modern computation increasingly depends on transformations whose authority and consequences are not fully captured by source code, binaries, policies, or configuration files alone. Its nearest external comparison fields are formal, security, governance. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counter-pressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] George C. Necula. “Proof-Carrying Code.” *POPL ’97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E2**] Xavier Leroy. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E3**] Patrick Cousot and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” *POPL ’77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

[**E4**] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html **Relationship:** Predecessor specification discipline for state machines, safety, liveness, and refinement.

[**E5**] Jerome H. Saltzer and Michael D. Schroeder. “The Protection of Information in Computer Systems.” *Proceedings of the IEEE* 63(9), 1975, pp. 1278-1308. DOI: 10.1109/PROC.1975.9939. https://doi.org/10.1109/PROC.1975.9939 **Relationship:** Foundational protection principles used as a comparison point for least authority, mediation, and inspectability.

[**E6**] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD dissertation, Johns Hopkins University, 2006. http://www.erights.org/talks/thesis/ **Relationship:** Predecessor object-capability treatment of authority and compositional security.

[**E7**] Gerwin Klein et al. “seL4: Formal Verification of an OS Kernel.” *SOSP ’09*, 2009, pp. 207-220. DOI: 10.1145/1629575.1629596. https://doi.org/10.1145/1629575.1629596 **Relationship:** Predecessor for machine-checked implementation correspondence and capability-based kernel design.

Ableman corpus

[**A1**] Adam Ableman. *Governed Semantic Compilation.* Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;
source SHA-256
`5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Hybrid Semantic Compiler Architectures.* Ableman Corpus current edition; work identity `PAPER.PUB.21228038`;
source SHA-256
`EBCD6DDDA9B547639AEBED79FB68069473CF1248E41C527B48C9624BBE2C0496`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Admissibility Engineering.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-F1E33B5E973B0900`;
source SHA-256
`CDC80271268802CA5CB4DE3624B992FB2FF07D677BF945C4B349BD307C4C2A6E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Everything Is a Compiler.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-30FB8F95BD990A47`;
source SHA-256
`FE0F88A1F125317285BE4378E5F27ACC9FEB83ADE3FF329A3DA9F9CB026DF9A9`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Effect Sovereignty.* Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality.* Ableman Corpus current edition; work identity `PAPER.PUB.18601631`;
source SHA-256
`6DE46F821594AB11BFB7F8373368C452BD89E5BD4EB172B88D9467E87CF74C33`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *The Integrity Enforcement Layer.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-505307E9E6571B63`;
source SHA-256
`F858AB32E0937B796353CAE573DD4183FDF4F179A3E1E7E40C4E0C6BF1AB0018`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.21095303